第35课 DFS入门

> 本讲义配合课堂例题讲解使用，代码可直接在 Dev-C++ 中运行验证。

五、课堂例题

例题 1：输出 1~3 的全排列（入门核心题）

题目描述

输出数字 1、2、3 的所有排列，每行一个排列，数字之间用空格隔开。要求按字典序输出。

预期输出

```
1 2 3
1 3 2
2 1 3
2 3 1
3 1 2
3 2 1
```

讲解思路

> **第一步——想清楚递归在干什么**
>
> 我们写一个函数 `dfs(step)`，表示"现在要填第 `step` 个位置"。
>
> - 如果 3 个位置都填满了（`step > 3`），就输出结果。
> - 否则，从 1 到 3 逐个尝试：如果这个数字还没被用过，就选它、标记它、去填下一个位置，填完回来后再取消标记。
>
> **第二步——需要哪些"工具"?**
>
> - `arr[]`：记录每个位置放了什么数字（就像 3 个空格子）。
> - `used[]`：记录每个数字有没有被用过（就像衣服上的标签，贴了标签的不能再选）。
>
> **第三步——回溯是什么?**
>
> 递归回来后，把 `used[i]` 从 `true` 改回 `false`，就像把衣服放回衣柜。这一步叫**回溯**。

完整代码

```
#include <iostream>
using namespace std;

int arr[4];      // arr[1], arr[2], arr[3] 分别存放第1、2、3个位置的数字
bool used[4];    // used[i] = true 表示数字 i 已经被选过了

// dfs(step) 表示：现在正在填第 step 个位置
void dfs(int step) {
    // 递归出口：3个位置都填满了，输出结果
    if (step > 3) {
        cout << arr[1] << " " << arr[2] << " " << arr[3] << endl;
        return;
    }

    // 从 1 到 3，逐个尝试每个数字
    for (int i = 1; i <= 3; i++) {
        if (!used[i]) {          // 如果数字 i 还没被用过
            arr[step] = i;       // 把 i 放在第 step 个位置
            used[i] = true;      // 标记：数字 i 已经用了

            dfs(step + 1);       // 递归去填下一个位置

            used[i] = false;     // 回溯：把标记擦掉，i 可以重新被选
            // arr[step] 不用特意清除，下次循环会覆盖
        }
    }
}

int main() {
    dfs(1);  // 从第 1 个位置开始填
    return 0;
}
```

逐行讲解要点

代码行	对应比喻
`dfs(step)`	小明正在填第 step 个格子
`if (step > 3)`	3 个格子填满了，拍照记录这种排法
`for (int i = 1; i <= 3; i++)`	逐个试试衣柜里的衣服
`if (!used[i])`	这件衣服还挂在衣柜里，没被拿走
`arr[step] = i`	把衣服拿出来穿上（放进格子）
`used[i] = true`	在衣柜的位置贴个"已取走"的标签
`dfs(step + 1)`	去填下一个格子
`used[i] = false`	**回溯**！把衣服放回去，撕掉标签

> **课堂演示**：老师在电脑上运行程序，带同学对照搜索树验证输出顺序。

例题 2：输出 1\~N 的全排列（泛化版）

题目描述

输入一个正整数 N（1 ≤ N ≤ 8），输出 1\~N 的所有全排列，每行一个排列，数字之间用空格隔开，按字典序输出。

输入样例

```
4
```

输出样例（前几行）

```
1 2 3 4
1 2 4 3
1 3 2 4
1 3 4 2
1 4 2 3
1 4 3 2
2 1 3 4
...（共24行）
```

讲解思路

> 和例题 1 几乎一样，只是把"3"换成了变量 N。让同学们感受**代码复用**的威力——改一个数字就能解决更大的问题！

完整代码

```
#include <iostream>
using namespace std;

int n;
int arr[10];       // 最多排8个数，开大一点
bool used[10];

void dfs(int step) {
    if (step > n) {
        // 输出当前排列
        for (int j = 1; j <= n; j++) {
            if (j > 1) cout << " ";
            cout << arr[j];
        }
        cout << endl;
        return;
    }

    for (int i = 1; i <= n; i++) {
        if (!used[i]) {
            arr[step] = i;
            used[i] = true;

            dfs(step + 1);

            used[i] = false;   // 回溯
        }
    }
}

int main() {
    cin >> n;
    dfs(1);
    return 0;
}
```

课堂互动

- 让同学先猜：N = 4 时一共输出多少行？（24 行）

- N = 5 呢？（120 行）
- "计算机跑 N = 8 要多久？"运行给他们看——一瞬间！（8! = 40320 种，计算机很快）
- "如果 N = 10 呢？"（10! = 3628800，可能要等一会了——引出 DFS 的时间复杂度概念）

例题 3：从 N 个数中选 M 个排列

题目描述

输入正整数 N 和 M（1 ≤ M ≤ N ≤ 8），输出从 1\~N 中选 M 个数的所有排列（顺序不同算不同排列），按字典序输出。

输入样例

```
4 2
```

输出样例

```
1 2
1 3
1 4
2 1
2 3
2 4
3 1
3 2
3 4
4 1
4 2
4 3
```

讲解思路

> 和全排列唯一的区别：递归出口从 `step > n` 变成 `step > m` ——填满 M 个格子就输出。
> 比喻：原来衣柜有 4 件衣服，要全穿上；现在只要从 4 件里选 2 件穿，穿的**顺序不同**算不同搭配。

完整代码

```
#include <iostream>
using namespace std;

int n, m;
int arr[10];
bool used[10];

void dfs(int step) {
    if (step > m) {    // 只需要填 m 个位置
        for (int j = 1; j <= m; j++) {
            if (j > 1) cout << " ";
            cout << arr[j];
        }
        cout << endl;
        return;
    }

    for (int i = 1; i <= n; i++) {
        if (!used[i]) {
            arr[step] = i;
            used[i] = true;

            dfs(step + 1);

            used[i] = false;
        }
    }
}

int main() {
    cin >> n >> m;
    dfs(1);
    return 0;
}
```

教学提示

> 引导同学观察：三道例题的代码**框架几乎一模一样**！DFS 的模板就是这样，学会一个，后面都是在这个模板上"改条件"。

附：例题源代码与运行结果

例题 1：Permutations of 1~3 (Core Intro)

```
// Output all permutations of 1, 2, 3 in lexicographic order.
// Demonstrates the DFS template: select, mark, recurse, backtrack.
#include <iostream>
using namespace std;

int arr[4];      // arr[1..3]: the number placed at each position
bool used[4];    // used[i] = true means number i is already chosen

// dfs(step): fill the step-th position
void dfs(int step) {
    // Base case: all 3 positions are filled
    if (step > 3) {
        cout << arr[1] << " " << arr[2] << " " << arr[3] << endl;
        return;
    }

    // Try each number from 1 to 3
    for (int i = 1; i <= 3; i++) {
        if (!used[i]) {          // If number i hasn't been used
            arr[step] = i;       // Place i at position step
            used[i] = true;      // Mark i as used

            dfs(step + 1);       // Recurse to fill the next position

            used[i] = false;     // Backtrack: unmark i
        }
    }
}

int main() {
    dfs(1);  // Start filling from position 1
    return 0;
}
```

Expected Output

```
1 2 3
1 3 2
2 1 3
2 3 1
3 1 2
3 2 1
```

例题 2：Permutations of 1~N (Generalized)

```
// Read N (1 <= N <= 8), output all permutations of 1~N in lexicographic order.

/*
#include <iostream>
using namespace std;

int n;
int arr[10];
bool used[10];

void dfs(int step) {
    if (step > n) {
        for (int j = 1; j <= n; j++) {
            if (j > 1) cout << " ";
            cout << arr[j];
        }
        cout << endl;
        return;
    }

    for (int i = 1; i <= n; i++) {
        if (!used[i]) {
            arr[step] = i;
            used[i] = true;

            dfs(step + 1);

            used[i] = false;  // Backtrack
        }
    }
}

int main() {
    cin >> n;
    dfs(1);
    return 0;
}
*/
```

Sample Input

```
4

Expected Output (first 6 of 24 lines):
1 2 3 4
1 2 4 3
1 3 2 4
1 3 4 2
1 4 2 3
1 4 3 2
... (24 lines total)
```

例题 3：Choose M from N (Partial Permutation)

```
// Read N and M (1 <= M <= N <= 8). Output all ways to arrange M numbers
// chosen from 1~N. Order matters. Output in lexicographic order.

/*
#include <iostream>
using namespace std;

int n, m;
int arr[10];
bool used[10];

void dfs(int step) {
    if (step > m) {        // Only need to fill m positions
        for (int j = 1; j <= m; j++) {
            if (j > 1) cout << " ";
            cout << arr[j];
        }
        cout << endl;
        return;
    }

    for (int i = 1; i <= n; i++) {
        if (!used[i]) {
            arr[step] = i;
            used[i] = true;

            dfs(step + 1);

            used[i] = false;  // Backtrack
        }
    }
}

int main() {
    cin >> n >> m;
    dfs(1);
    return 0;
}
*/
```

Sample Input

```
4 2
```

Expected Output

```
1 2
1 3
1 4
2 1
2 3
2 4
3 1
3 2
3 4
4 1
4 2
4 3
```